

Augmentation On A Clean Split

Two identical 100-epoch runs differing in one thing: whether the training images were randomly flipped, rotated and brightness-shifted. The augmented model classified all 715 held-out castings correctly. The unaugmented one memorised its training set instead.

train 6,569 images validation 715 epochs 100 batch 16 lr 1e-4, StepLR 35/0.1

RESULT

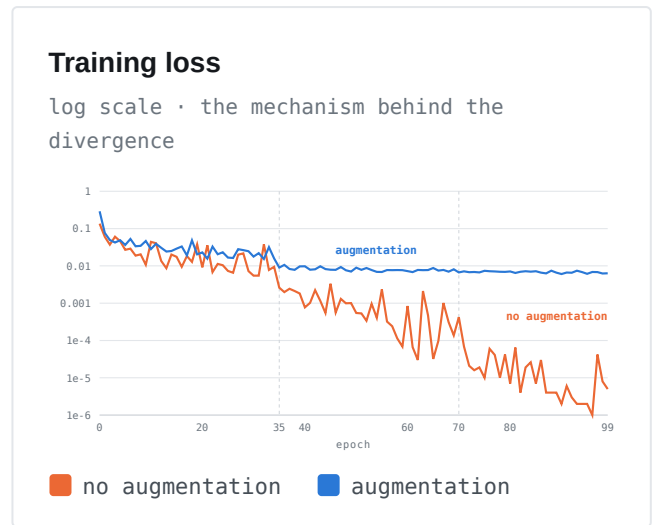
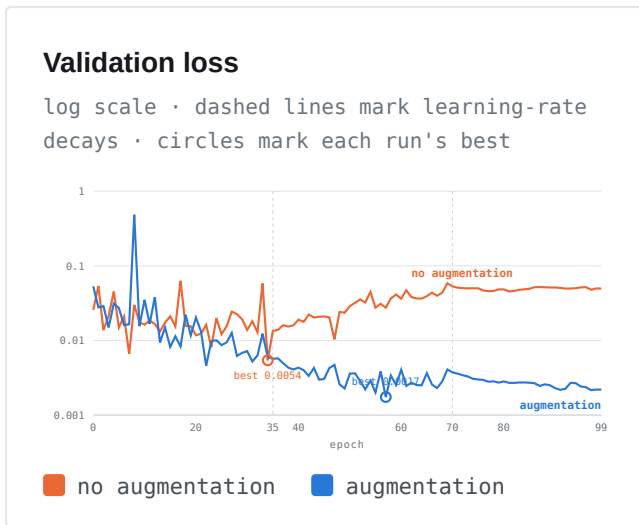
715 of 715, on a split
verified free of duplicates.

NO AUGMENTATION	WITH AUGMENTATION
99.72%	100%
2 defects missed · 0 false alarms	0 missed · 0 false alarms

MEASURE	NO AUGMENTATION	AUGMENTED
Accuracy	99.72%	100.00%
Errors (of 715)	2	0
Defects missed	2	0
Good parts wrongly rejected	0	0
Defect recall	0.9956	1.0000
Best validation loss	0.00543 (ep 34)	0.00175 (ep 57)
Final validation loss	0.04964	0.00220
Degradation, best → final	9.1× worse	1.3×
Final training loss	0.000005	0.006321

THE EVIDENCE

The accuracy difference is noise. The loss curves are not.



The two runs track each other until roughly epoch 34, then separate decisively. Without augmentation, validation loss bottoms out at 0.0054 and then climbs for the remaining 66 epochs, finishing **9.1× worse than its own best**. With augmentation it keeps falling to 0.0017 at epoch 57 and stays there, ending 1.3× off its best — essentially flat.

The right-hand chart shows why. The unaugmented model drives its training loss to **0.000005** — it has effectively memorised all 6,569 training images. The augmented model never gets below 0.006, because it never sees the same image twice. It cannot memorise, so it has to learn what actually distinguishes a defective casting.

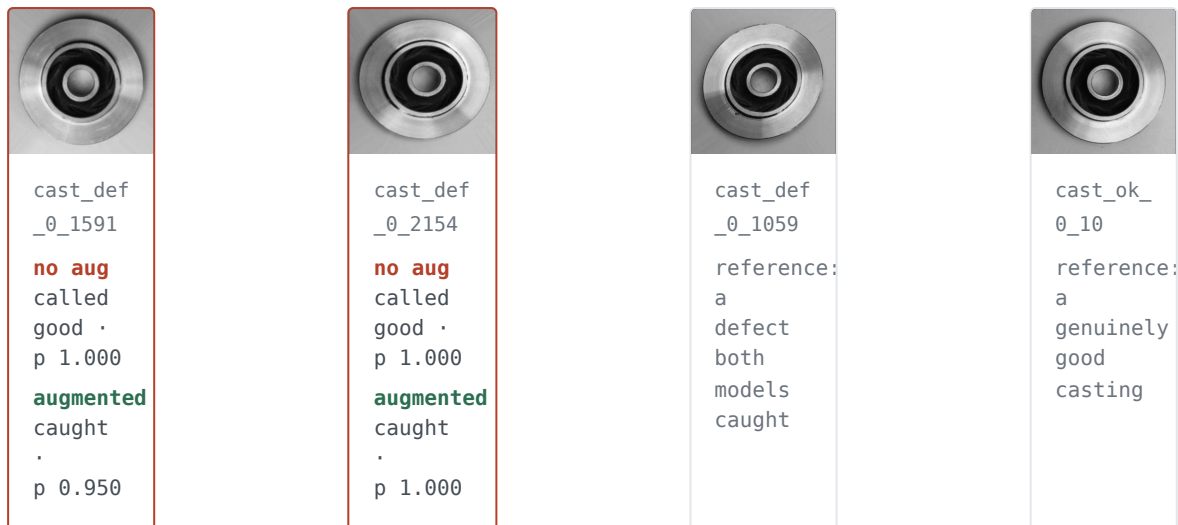
The single spike at epoch 8 in the augmented run is one unstable epoch, recovered immediately. Early augmented training is noisier; that is expected and not a fault.

EPOCH	TRAIN LOSS	VAL LOSS	TRAIN LOSS (AUG)	VAL LOSS (AUG)
0	0.13529	0.02576	0.29119	0.05322
5	0.02690	0.01492	0.03627	0.02723
10	0.04421	0.01631	0.02826	0.03507
20	0.00914	0.01175	0.02303	0.02034
30	0.00546	0.01384	0.01770	0.00722
34	0.00941	0.00543	0.01590	0.00604
40	0.00077	0.01903	0.00977	0.00432
50	0.00055	0.02887	0.00893	0.00361
57	0.00024	0.02755	0.00770	0.00175
60	0.00084	0.03633	0.00725	0.00407
70	0.00042	0.05301	0.00668	0.00375
80	0.00001	0.04830	0.00710	0.00283
90	0.00000	0.05142	0.00606	0.00230
99	0.00001	0.04964	0.00632	0.00220

THE TWO IT MISSED

Both errors, and the unaugmented

model was certain about each.



The confidences are the point. The unaugmented model called both of these good at a probability of **1.000** — not hesitant, not borderline, completely certain and completely wrong. The augmented model caught both, at 0.950 and 1.000.

Compare that with the same model earlier in training. In a 15-epoch run these two castings were also the only failures, but at confidences of 0.94 and 0.79 — the model was unsure, and a confidence threshold would have flagged them for a human. After 100 epochs of memorising the training set, that uncertainty was gone. Overfitting did not just make the model wrong; it made the model *sure*, which removes the one safety net a borderline case has.

The defects themselves are subtle. Against the two reference castings on the right, the failures show no obvious rim irregularity at 300×300 after greyscale conversion and resizing. Whatever marks them defective is faint, possibly outside the frame, or arguably a labelling edge case — which is exactly why a model that hedges is worth more here than one that does not.

WHAT THE NUMBERS MEAN

Plain definitions, with this run's figures.

accuracy	Share of castings classified correctly. 100% means all 715 were right. It is the easiest number to quote and the least informative — it treats a missed defect and a wrongly-scrapped good part as the same mistake, when on a production line they cost very different amounts.
defects missed (false negatives)	Defective castings the model passed as good. These reach the customer. The unaugmented model missed 2; the augmented model missed none. On a safety-relevant part this is the number that matters most.
false alarms (false positives)	Good castings the model rejected. These are scrapped needlessly — wasted material, not escaped risk. Both models had zero.
precision	Of everything the model called defective, the share that really was. 1.0000 for both — nothing was rejected without cause.
recall	Of everything genuinely defective, the share the model caught. 0.9956 without augmentation, 1.0000 with. Precision and recall pull against each other: a model that flags everything has perfect recall and terrible precision. Reporting only one hides which failure you have.
training loss	How wrong the model is on the images it is learning from. Falls as it learns. Reaching near-zero is not success — it can mean the model has memorised the answers rather than learned the pattern, which is exactly what 0.000005 indicates here.
validation loss	How wrong the model is on images it has never seen, weighted by how confident it was. This is the honest measure of generalisation. It is more sensitive than accuracy: a model can keep making the same correct calls while becoming badly overconfident, and only loss will show it.

overfitting

Training loss falling while validation loss rises. The model is getting better at its training set and worse at everything else. **Visible here from epoch 34 onward in the unaugmented run**, and absent in the augmented one.

epoch

One complete pass through the training set. 100 epochs means each of the 6,569 images was shown 100 times — though with augmentation, never in the same form twice.

learning rate & StepLR 35/0.1

How large a correction the model makes after each batch. Started at 0.0001 and was cut 10-fold at epochs 35 and 70, so the model takes big steps early and fine ones late. Without that decay the final weights bounce around rather than settle — which matters because this library saves the last epoch, not the best one.

THE SPLIT HAD TO BE FIXED FIRST

8.95% of the published test set was already in the training set.

A hash check of every image found **64 validation images byte-identical to training images** — all in the `ok_front` class. That is a flaw in the published casting dataset, not in the pipeline: its `train` and `test` directories genuinely share files.

Scoring on images the model has already learned inflates the result. The fix was to remove the 64 duplicates from **training** rather than from validation, so the benchmark stays the full published 715-image test set while the training set drops to 6,569. Both runs here use that corrected split, and `check_leakage.py` reports it clean.

Worth stating in any writeup, because published results above 98% on this dataset are common and the duplication is not widely noted.

WHAT THIS DOES AND DOESN'T PROVE

The accuracy is not the finding. The loss curve is.

0 errors versus 2 is inside the noise. On 715 samples, a 95% interval around 0 errors runs to 4, and around 2 errors from 1 to 7. Those overlap. On accuracy alone you cannot claim augmentation is better — rerun with a different random seed and the counts could swap.

The loss comparison is not inside the noise. A $9.1\times$ degradation against a $1.3\times$ one, with training loss separating by three orders of magnitude, is a large effect in a continuous measure, in the direction theory predicts, with the mechanism visible. That is the defensible claim.

The augmented model's best was also better. 0.00175 against 0.00543 — $3.1\times$ lower at each run's own best epoch, not just at the end. So augmentation did not merely prevent decay; it reached a genuinely better model.

This dataset is not hard. Consistent lighting, centred framing, one part type. 100% here does not imply 100% on a real line with varying conditions — but the augmented model is the one with a reason to hold up when those conditions change, which is the practical argument for using it.

Still untested: actual robustness. Both models were evaluated on images captured under the same conditions as training. Testing them against deliberately rotated and brightness-shifted validation images would measure the robustness augmentation is supposed to buy. That needs no retraining — both sets of weights are saved.

METHOD

Everything needed to reproduce this.

AlexNet pretrained on ImageNet, fine-tuned for two-class casting inspection. Images converted to greyscale, replicated to 3 channels, resized to 224×224 . Adam at $1e-4$, batch 16, cross-entropy, StepLR decaying 10-fold every 35 epochs, 100 epochs, CUDA.

The augmented run adds, to the training set only:

```
RandomHorizontalFlip(0.5), RandomVerticalFlip(0.5),
```

`RandomRotation(20°)`, `ColorJitter(brightness 0.2, contrast 0.2)`.

Flips and rotation because a circular casting can arrive at any orientation; jitter because shop-floor lighting drifts. No cropping — a defect near the rim must not be cut out of frame, which would change the label rather than vary the image.

The validation set is never augmented. Augmenting it would change the target between runs and make the two incomparable.

Both runs wrote `results.json`, `summary.csv`, `predictions.csv` and their own weights. Every figure on this page comes from those files; none was transcribed by hand.

gfitz01/Electronics-Manufacturing-Capstone-26-27
results/alexnet-20260914-212525 (no augmentation) · results/alexnet-aug-20260914-225622 (augmented)
6,569 train / 715 validation · duplicates removed from training · leakage check passing